

Nota algunas parecen ser boludeces pero al no tener ide son muy olvidables

Orden y ArrayList

- No confundir `ArrayList<>()` con el arreglo vanilla de Java $\rightarrow []$
 - El arreglo vanilla de Java tiene el metodo `length` para obtener la longitud, no existe el metodo `size()`
 - No olvidarse de checkear `idx > 0` si se consulta
 - Cuando se ordena el arreglo vanilla, hay que hacer una copia con su dimension real porque sino estarias desreferenciando NULLs y va a arrojar una `NullPointerException`
- De la siguiente forma (en pseudo)

```
private static final int DIM_INICIAL = 10;
private Cosa[] arrCosas = new Cosa[DIM_INICIAL];
private int dim_real = 0;

static Cosa[] getCopu(int dim) {
    return Arrays.copyOf(arrCosas, dim);
}

add(Cosa cosa) {
    if (arrCosas.length == dim_real) {
        arrCosas = getCopu(dim_real + DIM_INICIAL);
    }
    arrCosas[dim_real++] = cosa;
}

sorter() {
    Cosa[] cosasOrdenadas = Arrays.copyOf(arrCosas);
    // Suponiendo que Cosa es comparable
    Arrays.sort(cosasOrdenadas, Comparator.naturalOrder());
    // Notemos que el .sort no retorna un arreglo
    return cosasOrdenadas;
}
```

Clases abstractas

- Los constructores no pueden ser publicos

Generales

- Checkear de retornar bien los valores. **Me paso varias veces** cuando hay que retornar `this`, pongo bien el valor de retorno pero me olvido de devolverlo
- No olvidarse de implementar metodos aunque sean getters prestarle mucha atencion a los metodos que se llaman en el test

Enums

- **No leakear atributos**: Hacer todas las operaciones posibles dentro de los enums, no hacer getters de los atributos porque no corresponde

- Importante tener en cuenta `.values()` que retorna arreglo con las instancias de todos los enums ordenadas por `ordinal()`

```
public enum Level {

    // Es mejor hacer Level.values()[ordinal() + 1]
    // porque si se quiere agregar un nivel intermedio, habria que cambiar todo
    // pero no tuve en cuenta .values() la verdad (devuelve un arreglo con las
    // instancias de los enums)
    ENTRY(0.1, 100) {

        private Level levelUp() {
            return GOLD;
        }

    }, GOLD(0.2, 600) {

        private Level levelUp() {
            return PREMIUM;
        }

    }, PREMIUM(0.5, 0) {

        private Level levelUp() {
            return this;
        }

    };

    private final double multiplier;
    private final int limit;

    private Level(double multiplier, int limit) {
        this.multiplier = multiplier;
        this.limit = limit;
    }

    private Level updateLevel(int points, int limit) {
        if (points >= limit) {
            return levelUp();
        }
        return this;
    }

    public Level getUpdatedLevel(int points) {
        return updateLevel(points, limit);
    }

    public int getCorrespondingPointsOfPurchase(int purchase) {
        return (int) (purchase * multiplier);
    }

}
```